Analyse des émotions dans des textes arabes

De la compréhension métier à la préparation des données

Projet d'intelligence artificielle et de fouille de données

31 octobre 2026

Ce rapport présente la conception d'un agent IA de collecte sur le Web, l'exploration d'un corpus de textes arabes et sa préparation pour une analyse des émotions par thème.

Résumé exécutif

Nous avons conçu un agent IA qui recherche des pages Web publiques à partir d'une requête en arabe, extrait des segments arabophones, puis utilise un modèle Gemini pour filtrer les textes pertinents et proposer une étiquette de thème et d'émotion. Les données structurées sont exportées en CSV ; un manifeste accompagne les nouvelles collectes afin de relier chaque observation à son URL, à sa requête et à ses métadonnées de traitement.

Le corpus exploré dans ce projet comprend 50 050 lignes au départ. Après normalisation et suppression de 995 textes répétés, le jeu préparé comporte 49 055 textes répartis en six thèmes et cinq émotions. Une séparation stratifiée reproductible a produit 34 327 observations d'entraînement, 7 347 de validation et 7 381 de test. Le travail couvre les phases de compréhension métier, collecte, compréhension des données et préparation ; il constitue une base prête pour une prochaine expérimentation de classification.

Question de recherche : comment les émotions annotées varient-elles selon le thème abordé dans les textes arabes étudiés ?

1 Compréhension métier

1.1 Contexte et problème à résoudre

Les conversations publiées sur Internet contiennent des réactions et des opinions sur des sujets variés. Leur lecture manuelle rend difficile la comparaison systématique de grands volumes de textes. Le projet organise ces contenus dans un corpus structuré afin de décrire les émotions qui leur sont associées et leur variation selon le thème.

Le périmètre observé regroupe six thèmes : sport, économie, politique, religion, technologie et santé. Les émotions retenues sont la joie (`joy`), la colère (`anger`), la tristesse (`sadness`), l'espoir (`hope`) et le neutre (`neutral`).

1.2 Pourquoi la fouille de données et l'apprentissage automatique

La fouille de données permet de résumer les volumes, d'examiner les distributions et de faire apparaître des régularités dans les textes. Les techniques de traitement automatique du langage peuvent ensuite aider à classer de nouveaux commentaires à grande échelle. Dans ce projet, l'IA intervient déjà pour soutenir le filtrage et l'annotation initiale ; l'entraînement d'un classifieur dédié est une étape ultérieure, construite à partir des données préparées.

1.3 Objectifs et critères de réussite

— définir un corpus arabe organisé par thème et émotion ;

— automatiser la recherche, l'extraction, le filtrage et l'annotation initiale ;

— mesurer la structure, les distributions et la qualité des champs ;

— produire des données propres, documentées et réutilisables ;

— livrer des partitions reproductibles pour les prochaines analyses et expériences ML.

1.4 Plan de travail

Le projet suit une démarche inspirée de CRISP-DM : cadrer la question et les objectifs ; acquérir les pages et extraire les textes ; comprendre le corpus par des contrôles et statistiques descriptives ; transformer les données en un jeu cohérent et partitionné. Chaque phase produit des fichiers ou des informations utiles à la phase suivante.

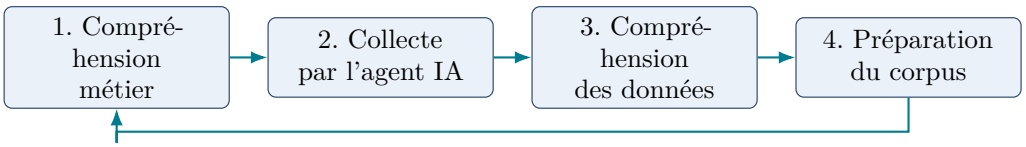


FIGURE 1 – Démarche générale du projet. La préparation alimente les analyses et les expérimentations futures.

2 Collecte des données

2.1 Sources et recherche sur le Web

L'agent utilise l'API Tavily pour rechercher des pages Web publiques. La demande est définie dans `prompt.txt`, avec une requête en arabe, la langue ciblée, les six thèmes et le nombre maximal de pages par exécution. La configuration courante limite la recherche à dix résultats et privilégie les contenus en arabe associés à la Tunisie.

La réponse du moteur peut inclure l'URL, le titre et le contenu extrait de chaque page. Le programme privilégie le contenu brut lorsqu'il est disponible, retire les éléments de mise en forme et les liens intégrés, puis découpe le texte en segments. Un filtre linguistique conserve les segments comportant suffisamment de caractères arabes ; les segments trop courts et les répétitions exactes sont écartés.

2.2 Agent IA et outils utilisés

L'agent est développé en Python. Tavily fournit la recherche et les résultats de pages. Gemini (modèle `gemini-2.5-flash`) assiste le filtrage de pertinence et attribue les champs `topic` et `emotion` selon un schéma structuré. Une température nulle et des catégories explicites rendent les réponses plus cohérentes d'une observation à l'autre. Les clés API sont chargées depuis le fichier local `.env` ; elles ne sont pas écrites dans les fichiers de données.

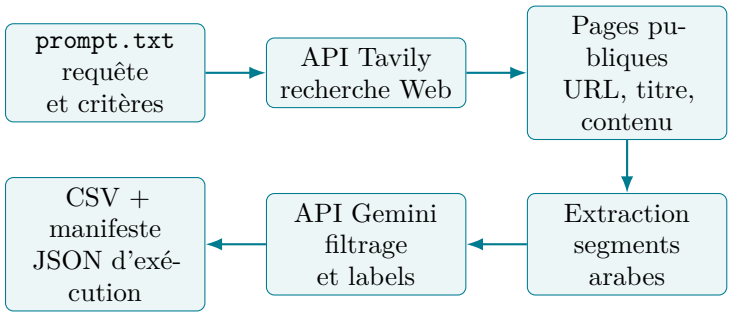


FIGURE 2 – Chaîne de collecte et d'annotation mise en œuvre dans `agent.py`.

2.3 Données brutes, données structurées et traçabilité

Les contenus retournés par la recherche constituent l'entrée brute de l'agent. Après extraction, chaque segment retenu est transmis au modèle pour déterminer sa pertinence, son thème et son émotion. Les lignes acceptées sont écrites dans `output/dataset.csv`, avec les quatre champs demandés : identifiant, texte, thème et émotion.

Pour assurer la traçabilité des exécutions, l'agent écrit également `output/dataset_sources.csv`, qui associe l'identifiant à l'URL et au titre de la page, à la requête, à l'heure UTC, à l'empreinte SHA-256 et au modèle d'annotation. Le résumé `output/collecte_run.json` consigne les paramètres et les volumes traités. Les étiquettes sont ainsi accompagnées de leur contexte de collecte dans les exécutions de l'agent.

3 Compréhension des données

3.1 Description du corpus

Le fichier de départ analysé, `output/arabic_topic_emotion_dataset_50000.csv`, contient 50 050 lignes. Son schéma est constitué de quatre colonnes : `id`, `text`, `topic` et `emotion`. Les contenus sont des textes courts en arabe et le traitement conserve les caractères arabes ainsi que les emojis.

TABLE 1 – Description des variables

Champ	Type	Rôle dans le projet
`id`	entier	Identifiant de la ligne pour le suivi et le rattachement au manifeste
`text`	texte	Contenu arabe traité par l'agent ; entrée textuelle pour un futur modèle
`topic`	catégorie	Thème du contenu ; variable de comparaison
`emotion`	catégorie	Émotion attribuée ; cible potentielle d'un classifieur

3.2 Exploration et qualité

Les contrôles structurels n'ont relevé aucune valeur manquante, aucun texte vide et aucune étiquette hors vocabulaire dans le fichier initial. La longueur des textes préparés varie de 32 à 81 caractères ; la médiane est de 57 et la moyenne de 56,84 caractères. La comparaison des textes a repéré 995 répétitions exactes, supprimées lors de la préparation.

Table 2 – Répartition du corpus préparé par thème

Thème	Effectif	Part
Économie	8 238	16,8%
Santé	8 092	16,5%
Politique	8 245	16,8%
Religion	8 165	16,6%
Sport	8 223	16,8%
Technologie	8 092	16,5%
Total	49 055	100%

Émotion	Effectif	Part
Colère	9 821	20,0%
Espoir	9 811	20,0%
Joie	9 807	20,0%
Neutre	9 822	20,0%
Tristesse	9 794	20,0%
Total	49 055	100%

La répartition des thèmes est proche d'un sixième du corpus par thème, et chacune des cinq émotions représente environ un cinquième des observations. Le fichier `croisement_theme_emotion.csv` présente les effectifs détaillés pour chaque couple thème–émotion et permet de poursuivre l'analyse comparative.

4 Préparation des données

4.1 Nettoyage et transformation

Le script `nettoyage.py` automatise les opérations suivantes :

1. vérification de la présence des quatre colonnes attendues ;
2. normalisation des espaces du texte et harmonisation des étiquettes en minuscules ;
3. suppression des textes vides et des catégories non prévues ;
4. déduplication exacte, insensible à la casse, en conservant la première occurrence ;
5. réattribution d'identifiants séquentiels ;
6. export des données nettoyées et séparation stratifiée avec la graine 42.

Les opérations préservent les caractères arabes, les emojis et le contenu textuel informatif. La stratification est faite selon les couples thème–émotion pour maintenir leur composition dans chaque partition.

4.2 Intégration et enrichissement

L'intégration se fait autour d'un schéma stable partagé par la collecte et le nettoyage. Les nouvelles sorties de l'agent sont prioritaires comme entrée du script de préparation ; le jeu archivé reste disponible comme source de repli. L'enrichissement consiste à ajouter aux textes les catégories de thème et d'émotion proposées par Gemini, puis à relier les identifiants aux métadonnées du manifeste. Les statistiques calculées à l'étape d'exploration documentent les volumes, les distributions, la longueur et les croisements de catégories.

4.3 Partitions et livrables

Table 3 – Séparation reproductible du corpus

Partition	Lignes	Proportion
Entraînement	34 327	70%
Validation	7 347	15%
Test	7 381	15%
Total	49 055	100%

Les fichiers livrés dans `projet ML/donnees/` sont `dataset_nettoye.csv`, `train.csv`, `validation.csv`, `test.csv`, `statistiques.json`, `exploration.json` et `croisement_theme_emotion.csv`. Les CSV utilisent UTF-8 avec BOM. Pour une première classification, `text` constitue l'entrée, `emotion` la cible et `id` reste un identifiant de suivi.

5 Reproduction du projet

Depuis la racine du dépôt, les dépendances Python sont `pandas`, `numpy`, `python-dotenv`, `tavily-python` et `google-genai`. Les deux clés API sont configurées localement dans `.env`. Les commandes principales sont :

```
python agent.py
python "projet ML/nettoyage.py"
python "projet ML/analyse_exploratoire.py"
```

La première commande recherche et annote les textes ; la deuxième nettoie et partitionne le CSV disponible ; la troisième regénère les statistiques d'exploration et le tableau croisé.

Conclusion

Le projet met en place une chaîne complète de préparation de données textuelles : le besoin métier guide la recherche, l'agent IA automatise la collecte et l'annotation initiale, l'exploration décrit le corpus, puis le nettoyage produit des partitions reproductibles. Ce livrable fournit une base organisée pour comparer les émotions entre thèmes et pour développer, dans une phase suivante, un modèle NLP évalué de manière structurée.